

REDIS

**In-Memory Database | Cache | Message
Broker(MQ)**

What is Redis?

Redis is an open source (BSD licensed) in-memory remote data structure store (database) that offers high performance, replication, and a unique data model. The full form of Redis is Remote Directory Server. Moreover, we can use it in multiple forms. Redis provides data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs, geospatial indexes, and streams.

What is Redis Used for?

- **1) In-Memory Database:** As an In-Memory database, We will get some empty memory to perform database operations. Moreover, it acts as [No-SQL database](#) and there are No Tables, No Sequences, No Joins concept. We can store data in the form of String, Hash Operations***, List, Set etc. In-built services will be available.
- **2) Cache:** We can also use Redis as a Cache to increase our application performance.
- **3) Message Broker(MQ):** Another use of Redis is as a Message Broker.

What is Redis Cache?

Redis Cache is nothing but a Cache Management feature offered by Redis. Redis is normally used as a cache to store repeatedly accessed data in memory so that the user can feel the better performance of the application. The Redis Cache offers various features like how long you want to keep data, and which data to remove first, and some other bright caching models.

What is the advantage of using Redis Cache in your application?

Like any other Caching Technique, Redis Cache also minimizes the number of network calls made by your application, which in return improves performance of the application as a whole. One request from an application to the DB is similar to one network call. We can also achieve the better scaling once we apply any caching mechanism in the application as the database can serve more calls in this case.

How does the Redis Cache work in the Application?

When we perform a DB retrieve operation via an Application, the Redis Cache stores the result in its cache. Further, when we perform the same retrieve operation, it returns the result from the cache itself and ignore the second call to database. Similarly, when we perform a DB update operation, the Redis Cache also updated the result in its cache. Needless to say, for delete operation also it deleted the data from the cache accordingly. In this way, there are no chances of getting incorrect data.

What are the important annotations to enable Redis Cache in the Application?

- **@EnableCaching**

We apply this annotation at the main class (starter class) of our application in order to tell Spring Container that we need a caching feature in our application.

- **@Cacheable**

@Cacheable is used to fetch (retrieve) data from the DB to the application and store in Redis Cache. We apply it on the methods that get (retrieve) data from DB. @Cacheable requires a return value of the method that adds or updates data in the cache.

- **@CachePut**

We use @CachePut in order to update data in the Redis Cache while there is any update of data in DB. We apply it on the methods that make modifications in DB.

- **@CacheEvict**

We use @CacheEvict in order to remove the data in the Redis Cache while there is any removal of data in DB. We apply it on the methods that delete data from DB. It can be used with void methods.

How to implement Redis Cache in Spring Boot Application?

- In order to implement Redis Cache using Spring Boot, we need to create one small application that will have CRUD operations in place. Then we will apply Redis Cache feature in Retrieve, Update and Delete Operations. However, we should not apply Cache feature in a Create operation as we will not get any benefit of it. Let's start working on 'How to implement Redis Cache in Spring Boot?' gitHub link below:
<https://github.com/devRameshwar/In-Memory-cache-Redis->

When to Use *@Caching* Annotation?

Suppose we want to apply multiple caching annotations on a particular method, let's try it with an example.

```
@CacheEvict("User")
```

```
@CacheEvict(value="Invoice", key="#id")
```

```
public void deleteInvoice(Integer invId) {
```

```
....
```

```
}
```

The above code would have a compilation error, since Java does not allow multiple annotations of the same type to be declared for a given method. In this case we should use `@Caching` annotation. For example, below code demonstrates the concept.

- **`@Caching(evict = {@CacheEvict("Invoice"),
@CacheEvict(value="Invoice", key="#invld") })
public void deleteInvoice(Integer invld) {
....
}`**

Similarly, if we have annotations of different type, we can group them using `@Caching` annotation for better readability. However, below code will not have any compilation error. For example,

```
@CacheEvict(value = "usersList", allEntries = true)  
@CachePut(value="user",key="#user.getUserId()")  
public User updateUser(User user) {  
  
    ....  
  
}
```

The above code can be converted as
below:

```
@Caching(evict={@CacheEvict(value="usersList",
allEntries = true)},
put={@CachePut(value="user",key="#user.getUserId()")})
public User updateUser(User user) {
    ....
}
```

How to Implement Conditional Caching using Annotations?

- Below Example demonstrates the concept of 'condition' parameter.
- **@CachePut(value="invoices",
condition="#invoice.amount>=2496")**
public String getInvoice(Invoice invoice) {
...
}

Now, let's see the concept of 'unless' parameter from the example below:

- **@CachePut(value="invoices",
 unless="#result.length()<24")
public String getInvoice(Invoice invoice) {
 ...
}**